

LeetCode 857: Minimum Cost to Hire K Workers

Authored by Arpit Raj, LNMIIT Jaipur | July 13, 2026

Problem Overview

We are given n workers, each with a specific **quality** and a minimum **wage** expectation. Our goal is to hire exactly K workers to form a paid group with the absolute minimum cost possible. This problem is frequently asked in interviews by top tech companies like **Arcesium** and **Google**.

1. The Need for a Ratio

Can we simply pay workers their minimum requested wages? **NO**.

The problem stipulates that every worker must be paid *strictly proportional* to their quality compared to other workers in the paid group. Therefore, every worker in the chosen group is paid according to a common base rate, let's call it R .

$$\text{Salary} = \text{Quality} \times R$$

2. Determining the Base Rate (R)

For any individual worker, the salary they receive must be greater than or equal to their minimum wage expectation:

$$\text{Quality} \times R \geq \text{min_wage} \rightarrow R \geq \text{min_wage} / \text{Quality}$$

This means $\text{wage} / \text{quality}$ is the minimum acceptable rate for a specific worker. To keep an entire group of K workers happy, the rate R used for the group must be the **maximum ratio** among all workers present in that group. Otherwise, the worker with the highest ratio requirement would not receive their minimum wage.

Optimization Strategy

Since the Total Salary is determined by the formula:

$$\text{Total Salary} = (\text{Total Quality Sum}) \times R$$

If we sort all workers by their ratio (ascending) and iterate through them, we are effectively fixing **R** to be the ratio of the current worker (which will be the highest in the group so far). Since **R** is fixed for the current iteration, our only way to minimize the **Total Salary** is to minimize the **Total Quality Sum**.

- We maintain a max-heap (Priority Queue) to track the qualities of the workers currently in our group.
- If our group size exceeds **K**, we remove the worker with the **highest quality** to keep the sum as low as possible.
- Whenever the group size is exactly **K**, we calculate the cost and update our minimum answer.

C++ Implementation

Below is the algorithm translated directly into C++, applying the ratio-sorting and priority queue logic:

```
class Solution {
public:
    double mincostToHireWorkers(vector<int>& quality, vector<int>& wage, int k)
    {
        int n = quality.size();
        vector<pair<double, int>> labour;

        for(int i = 0; i < n; i++) {
            double ratio = (double)wage[i] / quality[i]; // salary = quality *
            // r and it must be >= minwage
            labour.push_back({ratio, quality[i]});
        }

        sort(labour.begin(),
labour.end()); // kyunki r * totqualitysum minimize krna he to start with least
ratio

        double ans = DBL_MAX;
        long long tot = 0;
```

```
priority_queue<int> maxh;

for(auto& bihari : labour) {
    double rat = bihari.first;
    int q = bihari.second;

    maxh.push(q);
    tot += q;

    if(maxh.size() > k) {
        tot -= maxh.top(); // ratio fixed he to total quality sum
        minimize krne ke liye sabse bada quality hata do
        maxh.pop();
    }

    if(maxh.size() == k) {
        ans = min(ans, rat * tot);
    }
}

return ans;
}
```

```
};
```